Ανάλυση και Κατηγοριοποίηση Δεδομένων

Sonar, Mines vs. Rocks

Ψηφιακά Συστήματα
Πανεπιστήμιο Πειραιά
Αθήνα Αττική Ελλάδα
harisantoniadi@gmail.com

Περίληψη

Σε αυτή την τεχνική αναφορά θα γίνει περιγραφή της λογικής και τεχνικής που ακολουθήθηκε με στόχο την κατηγοριοποίηση των δοθέντων δεδομένων. Συγκεκριμένα, το σύνολο δεδομένων στο οποίο αναφέρεται είναι το "Sonar, Mines vs. Rocks Dataset". Στα σχετικά αρχεία περιέχονται πληροφορίες που αφορούν την διάδοση των σημάτων Sonar (SOund Navigation Ranging). Το sonar αποτελεί μια τεχνική εξερεύνησης και χαρτογράφησης του ωκεανού, μέσω της μελέτης των ηχητικών κυμάτων κατά την υποβρύχια διάδοσή τους. Με την χρήση αυτής της τεχνικής οι θαλάσσιοι ερευνητές έχουν την δυνατότητα να αναπτύξουν ναυτικά γραφήματα (χάρτες) και να εντοπίσουν ενδεχόμενους υποβρύχιους κινδύνους ή διάφορα αντικείμενα (π.χ. ναυάγια) στην επιφάνεια της θάλασσας.

Πρόκειται για το σύνολο δεδομένων που χρησιμοποιήθηκε από τους Gorman και Sejnowski στη μελέτη τους σχετικά με την ταξινόμηση των sonar σημάτων χρησιμοποιώντας ένα νευρωνικό δίκτυο. Κύριος στόχος είναι η εκπαίδευση ενός δικτύου ώστε να είναι εφικτή η διάκριση των sonar σημάτων που διαδίδονται μέσω ενός μεταλλικού κυλίνδρου και εκείνων που διαδίδονται μέσω μιας σχεδόν κυλινδρικής πέτρας.

Μέσω της προγραμματιστικής γλώσσας Python και διαφόρων αλγορίθμων κατηγοριοποίησης, θα εξαχθούν δεδομένα, τα οποία στη συνέχεια θα αξιολογηθούν, ώστε τελικά να προκύψει ένα τελικό συμπέρασμα σχετικά με το σύνολο δεδομένων και τον διαχωρισμό των στοιχείων του.

CCS CONCEPTS

•Computing methodologies~Machine learning~Machine learning approaches~Classification and regression trees

KEYWORDS

Ανάλυση δεδομένων, classification, κατηγοριοποίηση, sonar, σύνολο δεδομένων, υποβρύχια ηχητικά κύματα, διάδοση σημάτων

ACM Reference format:

[1] Χάρις Αντωνιάδη. 2020. Ανάλυση και Κατηγοριοποίηση Δεδομένων: Sonar, Mines vs. Rocks. Ανάλυση Δεδομένων (4ο Εξάμηνο) 2η Εργασία Εξαμήνου. *Πανεπιστήμιο Πειραιά, Πειραιάς, Ελλάδα. 2 σελίδες.*

1 Εισαγωγή

Πρόκειται για ένα δυαδικό πρόβλημα κατηγοριοποίησης. Σκοπός της εργασίας αποτελεί η εφαρμογή των αλγορίθμων αξιολόγησης στο δοθέν σύνολο δεδομένων και η αξιολόγηση τους στη συνέχεια, έτσι ώστε να εξαχθούν ορθά συμπεράσματα για το ίδιο το σύνολο καθώς και για τις τεχνικές που εφαρμόστηκαν για να ληφθούν αυτά τα τελικά αποτελέσματα. Στην ουσία, επιθυμητό αποτέλεσμα είναι η εκπαίδευση των αλγορίθμων πάνω στο συγκεκριμένο σύνολο δεδομένων προκειμένου να μπορούν εν τέλει να λαμβάνουν οποιοδήποτε σύνολο τιμών, που να σχετίζεται με την μετάδοση των sonar σημάτων και να το κατατάσσουν σε μια από τις δύο υπάρχουσες κατηγορίες. Πιο συγκεκριμένα, στην προκείμενη περίπτωση αυτό σημαίνει πως όταν δοθούν σε κάποιον αλγόριθμο τιμές που προέκυψαν από τις δοκιμές υπό διαφορετικές γωνίες διάδοσης των σημάτων είτε μέσω ενός μεταλλικού κυλίνδρου είτε μέσω μίας πέτρας, εκείνος πρέπει να είναι σε θέση να κατηγοριοποιήσει αυτά τα δεδομένα σε μια από τις δύο αυτές κλάσεις (Μέταλλο/Mine ή Πέτρα/Rock).

2 Περιγραφή του συνόλου δεδομένων

Το αρχείο "sonar.mines" περιέχει 111 εγγραφές που λαμβάνονται από την διάδοση των σημάτων sonar, μέσω ενός μεταλλικού κυλίνδρου σε διάφορες γωνίες και υπό διάφορες συνθήκες. Παράλληλα, το αρχείο "sonar.rocks" περιέχει 97 εγγραφές, οι οποίες αυτή τη φορά λαμβάνονται

από τη διάδοση των σημάτων μέσω πετρωμάτων κάτω από τις αντίστοιχες συνθήκες. Το μεταδιδόμενο sonar σήμα αποτελεί ηχητικό σήμα (chirp), δηλαδή ένα σήμα του οποίου η συχνότητα αυξάνεται ή μειώνεται ανάλογα με το χρόνο. Το σύνολο δεδομένων περιέχει πληροφορίες για σήματα, που έχουν ληφθεί από μια ποικιλία διαφορετικών γωνιών διάδοσης (κυμαίνονται από 90ο για τον μεταλλικό κύλινδρο και 180ο για τις πέτρες). Κάθε εγγραφή αποτελείται από ένα σύνολο 60 αριθμών στο εύρος 0.0 έως 1.0. Ο κάθε ένας από αυτούς τους αριθμούς αντιπροσωπεύει την ενέργεια σε μία συγκεκριμένη ζώνη συχνοτήτων, ολοκληρωμένη για μία συγκεκριμένη χρονική περίοδο. Το άνοιγμα ολοκλήρωσης για υψηλότερες συχνότητες συμβαίνει αργότερα, καθώς αυτές οι συχνότητες μεταδίδονται πιο μετά κατά την διάρκεια του σήματος (chirp). Η ετικέτα που σχετίζεται με κάθε εγγραφή περιέχει το γράμμα "R" εάν το αντικείμενο είναι πέτρα και "M" εάν είναι μέταλλο (μεταλλικός κύλινδρος). Οι αριθμοί στις ετικέτες είναι τοποθετημένοι σε αυξανόμενη σειρά γωνίας διαστάσεων, αλλά δεν κωδικοποιούν άμεσα τη γωνία.

Το παρόν σύνολο δεδομένων μπορεί να χρησιμοποιηθεί με διάφορους τρόπους για να δοκιμαστεί η ταχύτητα καθώς και η ποιότητα μάθησης, η ικανότητα γενίκευσης ή ένας συνδυασμός όλων των παραπάνω. Οι Gorman και Sejnowski αναφέρουν δύο σειρές πειραμάτων. Η μια αποτελεί μια «ανεξάρτητη της γωνίας» σειρά, στην οποία χρησιμοποιείται ολόκληρο το σύνολο δεδομένων, χωρίς έλεγχο για τη γωνία διάδοσης και η δεύτερη αποτελεί μια «εξαρτώμενη από τη γωνία» σειρά κατά την οποία ελέγχθηκαν προσεκτικά τα σετ εκπαίδευσης και δοκιμών έτσι ώστε να είναι βέβαιο ότι κάθε ένα από αυτά περιείχε περιπτώσεις για κάθε γωνία διάδοσης σε κατάλληλες αναλογίες. Για τα ανεξάρτητα γωνίας πειράματα, το συνδυασμένο σύνολο των 208 εγγραφών χωρίστηκε τυχαία σε 13 διαχωριστικά σύνολα, με 16 περιπτώσεις στο κάθε ένα. Σε κάθε πείραμα, τα 12 από αυτά τα σύνολα χρησιμοποιήθηκαν σαν σετ εκπαίδευσης ενώ το 13ο προοριζόταν για δοκιμή. Το πείραμα επαναλαμβάνεται 13 φορές έτσι ώστε κάθε περίπτωση να εμφανίζεται μια φορά σαν μέρος του σετ δοκιμών. Η απόδοση αφορά κατά μέσο όρο ολόκληρο το σύνολο των 13 διαφορετικών δοκιμαστικών σετ, εκ των οποίων το καθένα εκτελείται 10 φορές.

Παρατηρήθηκε ότι αυτή η τυχαία διαίρεση του συνόλου δειγμάτων οδήγησε μάλλον άνιση απόδοση. Μερικά από τα χωρίσματα έδωσαν κακά αποτελέσματα, πιθανώς επειδή το σύνολο δοκιμών περιέχει ορισμένα δείγματα από γωνίες που υποεκπροσωπήθηκαν στο αντίστοιχο σετ προπόνησης. Αυτό το γεγονός παρακίνησε τους Gorman και Sejnowski να επινοήσουν ένα διαφορετικό σύνολο πειραμάτων στα οποία έκαναν μία απόπειρα για να εξισορροπήσουν τα σετ εκπαίδευσης και δοκιμής έτσι ώστε το καθένα να έχει αντιπροσωπευτικό αριθμό δειγμάτων για όλες τις γωνίες. Εφόσον, δεν υπήρχαν λεπτομερείς πληροφορίες που να αφορούν την γωνία στη βάση δεδομένων των δειγμάτων, τα 208 δείγματα χωρίστηκαν αρχικά σε συστάδες, χρησιμοποιώντας Ευκλείδεια μέθοδο μέτρησης 60 διαστάσεων. Κάθε μια από αυτές τις συστάδες διαιρέθηκε στη συνέχεια μεταξύ του σετ εκπαίδευσης 104 μελών και του σετ δοκιμών 104 μελών.

3 Προ-επεξεργασία δεδομένων

Τα δεδομένα εισόδου αποτελούν ένα κανονικοποιημένο φασματικό φάκελο που καλύπτει 60 δείγματα (ως εκ τούτου έχει ήδη υποστεί μεγάλη επεξεργασία και δεν αποτελεί μία χρονοσειρά αλλά το αποτέλεσμα αυτής). Επειδή λοιπόν το σύνολο δεδομένων έχει ήδη ομαλοποιηθεί, η μόνη προεπεξεργασία που πρέπει να κάνουμε είναι να κωδικοποιήσουμε τις κλάσεις (τύπου) συμβολοσειράς σε ακέραιους και να χωρίσουμε τα δεδομένα μας σε δεδομένα εκπαίδευσης και δοκιμών. Η χρήση του 20% των δεδομένων για δοκιμές μας αφήνει 166 δείγματα για εκπαίδευση καθώς και 42 δείγματα για τελική αξιολόγηση (παρουσιάζεται παρακάτω ο αντίστοιχος κώδικας).

Για να εισαχθούν τα δεδομένα και για να μπορέσει να γίνει χρήση των περιεχομένων του αρχείου στον κώδικα, θα πρέπει το αρχείο “sonar.all-data” να μετατραπεί σε μορφή “.csv”. Αυτό μπορεί να πραγματοποιηθεί με τον εξής τρόπο. Μετά την λήψη των δεδομένων από την αντίστοιχη σελίδα του Learning Repository, αυτά θα εισαχθούν στο Excel και ύστερα αποθηκευτούν με την κατάλληλη προέκταση -το εξαγόμενο αρχείο ονομάζεται “sonar.csv”. Για να είναι πιο κατανοητή η ανάλυση τους στη συνέχεια και να αποφευχθούν τυχόν παρεξηγήσεις σχετικά με τις γραμμές ή στήλες του αρχείου και τα αντίστοιχα περιεχόμενα/τιμές τους, προτείνεται η ονομασία των πρώτων 60 στηλών ως “attribute 1, attribute2…attribute 60” και της 61ης στήλης σαν “Class”. Επομένως, πλέον είναι γνωστό ότι στο σύνολο των 208 εγγραφών, η κάθε γραμμή του αρχείου αφορά μία από αυτές τις εγγραφές, ενώ ταυτόχρονα οι στήλες από 1 εώς 60 αφορούν την ενέργεια που καταναλώθηκε κατά τη διάδοση του σήματος κάτω από την συγκεκριμένη γωνία διάδοσης και τέλος η τελευταία στήλη δείχνει την κατηγορία στην οποία εμπίπτει η κάθε εγγραφή.

4 Αλγόριθμοι κατηγοριοποίησης, μεθοδολογία και πειραματική αξιολόγηση

Για να είναι εφικτή η ανάλυση του συνόλου δεδομένων σε διάφορα μέρη του κώδικα γίνεται εισαγωγή των κατάλληλων βιβλιοθηκών της Python.

Εισαγωγή της βιβλιοθήκης Pandas και διάβασμα του αρχείου sonar.csv

```
import pandas as pd

path = 'sample_data/sonar.csv'
dataset = pd.read_csv(path)
```

Εκτύπωση του μεγέθους του αρχείου (208 γραμμές – 61 στήλες)

```
print(dataset.shape)
```

Εκτύπωση του πλήθους εγγραφών που αντιστοιχούν σε μέταλλα και σε πέτρες αντίστοιχα, καθώς και το όνομα της στήλης που βρίσκονται

```
dataset['Class'].value_counts()
```

Εμφάνιση των πρώτων 5 εγγραφών

```
dataset.head()
```

Εμφανίζεται μια σύνοψη των περιεχομένων του αρχείου

```
dataset.info()
```

Εμφανίζεται μια στατιστική περιγραφή του συνόλου δεδομένων

```
dataset.describe()
```

Εισαγωγή της βιβλιοθήκης matplotlib.pyplot και δημιουργία ιστογραμμάτων που το κάθε ένα από αυτά αφορά μια από τις 60 στήλες του αρχείου

```
import matplotlib.pyplot as plt

dataset.hist(bins=50, figsize=(20,15))
plt.show()
```

Εμφανίζεται ένα διάγραμμα boxplots των συχνοτήτων όπως και διάγραμμα πυκνοτήτων ώστε η σύνοψη των δεδομένων να είναι λίγο πιο προσιτή

```
dataset.plot.box(figsize=(12,7), xticks=[])
plt.title('Boxplots of all frequency bins')
plt.xlabel('Frequency bin')
plt.ylabel('Power spectral density (normalized)')
plt.show()
```

```
dataset.plot(kind = 'density', subplots = True, layout = (8, 8), sharex = False, legend = False, fontsize = 1, figsize = (20,15))
plt.show()
```

Τα δεδομένα εισόδου δεν είναι ακατέργαστα δεδομένα sonar ή φασματογραφήματα αλλά φασματικοί φάκελοι. Έτσι γίνεται εμφάνιση διαγράμματος των φασματικών φακέλων για κάθε μια από τις τάξεις.

```
plt.figure(figsize=(8,5))
plt.plot(dataset[dataset['Class'] == 'R'].values[0][:-1], label='Rock', color='black')
plt.plot(dataset[dataset['Class'] == 'M'].values[0][:-
1], label='Metal', color='lightgray', linestyle='--')
plt.legend()
plt.title('Example of both classes')
plt.xlabel('Frequency bin')
plt.ylabel('Power spectral density (normalized)')
plt.tight_layout()
plt.show()
```

Εμφάνιση πολυμεταβλητής γραφικής παράστασης για καλύτερη κατανόηση των σχέσεων μεταξύ των χαρακτηριστικών

```
fig = plt.figure()
ax = fig.add_subplot(111)
cax = ax.matshow(dataset.corr(), vmin=-
1, vmax=1, interpolation='none')
fig.colorbar(cax)
plt.show()
```

Εισαγωγή του preprocessing από τη βιβλιοθήκη sklearn και κωδικοποίηση των δεδομένων εξόδου (“R” για πέτρα και “M” για μέταλλο)

```
from sklearn import preprocessing
encoder = preprocessing.LabelEncoder()
encoder.fit(y_original)
y_encoded = encoder.transform(y_original)
print(y_encoded)
```

Εισαγωγή του train_test_split από τη βιβλιοθήκη sklearn ώστε να γίνει χωρισμός του συνόλου δεδομένων σε σετ εκπαίδευσης και σετ δοκιμής (20%)

```
from sklearn.model_selection import train_test_split

seed = 7
X_train, X_test, y_train, y_test = train_test_split(X_original, y_encoded, test_size = .2, random_state = seed)
```

Στη συνέχεια γίνεται εισαγωγή όλων των κομματιών των βιβλιοθηκών που θα χρησιμοποιηθούν αργότερα κατά την ανάλυση των δεδομένων. Μετά την προετοιμασία δεδομένων, κύριος στόχος είναι η εύρεση ενός λειτουργικού μοντέλου αξιολογώντας ένα υποσύνολο αλγορίθμων μηχανικής μάθησης που είναι καλοί στην εκμετάλλευση της δομής του συνόλου δεδομένων. Οι τυπικές εργασίες αξιολόγησης περιλαμβάνουν τον καθορισμό των επιλογών δοκιμής όπως διασταυρούμενη επικύρωση και μέτρηση αξιολόγησης προς χρήση, το Spot checking μιας σειράς γραμμικών και μη γραμμικών αλγορίθμων μηχανικής μάθηση και την σύγκριση της εκτιμώμενης ακρίβειας των αλγορίθμων. Για την πραγματοποίηση αυτών των εργασιών, θα γίνει αξιολόγηση 6 διαφορετικών αλγοίθμων:

- Γραμμικοί αλγόριθμοι: Λογιστική παλινδρόμηση (LR) και Linear Discriminant Analysis (LDA)
- Μη γραμμικοί αλγόριθμοι: Δέντρα ταξινόμησης και παλινδρόμησης (CART), Μηχανή φορέα υποστήριξης (SVM), Gaussian Native Bayes (NB) και k-Nearest Neighbours (KNN)

Η μεταβλητή τυχαίου αριθμού επαναφέρεται πριν από κάθε εκτέλεση για να διασφαλιστεί ότι η αξιολόγηση κάθε αλγορίθμου πραγματοποιείται χρησιμοποιώντας τα ίδια διαχωριστικά δεδομένων. Διασφαλίζει ότι τα αποτελέσματα είναι άμεσα συγκρίσιμα.

Παρακάτω φαίνονται συγκεντρωτικά οι αλγόριθμοι που θα χρησιμοποιηθούν και στη συνέχεια θα αξιολογηθούν, για την ανάλυση και κατηγοριοποίηση του δοθέντος συνόλου δεδομένων.

Πραγματοποίηση spot cheking των αλγορίθμων

```
models = []
models.append(('LR', LogisticRegression()))
models.append(('LDA', LinearDiscriminantAnalysis()))
models.append(('KNN', KNeighborsClassifier()))
models.append(('CART', DecisionTreeClassifier()))
models.append(('NB', GaussianNB()))
models.append(('SVM', SVC()))
```

Αξιολόγηση του κάθε μοντέλου και εκτύπωση των αποτελεσμάτων, από τα οποία προκύπτει ότι το μοντέλο KNN είχε την υψηλότερη μέση ακρίβεια, ακολουθούμενο από LR και συνεπώς το μοντέλο KNN αξίζει περαιτέρω μελέτη.

```
results = []
names = []
for name, model in models:
    kfold = KFold(n_splits=num_folds, random_state=seed)
    cv_results = cross_val_score(model, X_train, y_train, cv=kfold, scoring=scoring)
    results.append(cv_results)
    names.append(name)
    msg = "%s: %f (%f)" % (name, cv_results.mean(), cv_results.std())
    print(msg)
```

Σύγκριση των αλγορίθμων και εκτύπωση του αντίστοιχου διαγράμματος

```
fig = plt.figure()
fig.suptitle('Algorithm Comparison')
ax = fig.add_subplot(111)
plt.boxplot(results)
ax.set_xticklabels(names)
fig.set_size_inches(8,6)
plt.show()
```

Παρατηρούμε ότι οι διαφορετικές κλίμακες των δεδομένων εκπαίδευσης ενδέχεται να έχουν αρνητική επίδραση στην ακρίβεια των αλγορίθμων. Για αυτό οι αλγόριθμοι θα αξιολογηθούν χρησιμοποιώντας ένα τυποποιημένο σύνολο δεδομένων. Τυποποίηση σημαίνει ότι τα δεδομένα μετατρέπονται έτσι ώστε κάθε χαρακτηριστικό να έχει μέση τιμή μηδέν και τυπική απόκλιση ενός. Παρακάτω φαίνονται οι εντολές που θα ακολουθηθούν και στη συνέχεια οι εντολές για την εκτύπωση των αντίστοιχων αποτελεσμάτων.

```
pipelines = []
pipelines.append(('ScaledLR', Pipeline([('Scaler', StandardScaler()), ('LR', LogisticRegression())])))
```

```
pipelines.append(('ScaledLDA', Pipeline([('S
caler', StandardScaler()), ('LDA', LinearDis
criminantAnalysis())])))
pipelines.append(('ScaledKNN', Pipeline([('S
caler', StandardScaler()), ('KNN', KNeighbor
sClassifier())])))
pipelines.append(('ScaledCART', Pipeline([('
Scaler', StandardScaler()), ('CART', Decisio
nTreeClassifier())])))
pipelines.append(('ScaledNB', Pipeline([('Sc
aler', StandardScaler()), ('NB', GaussianNB(
))])))
pipelines.append(('ScaledSVM', Pipeline([('S
caler', StandardScaler()), ('SVM', SVC())]))
)
results = []
names = []
for name, model in pipelines:
    kfold = KFold(n_splits=num_folds, random
_state=seed)
    cv_results = cross_val_score(model, X_tr
ain, y_train, cv=kfold, scoring=scoring)
    results.append(cv_results)
    names.append(name)
    msg = "%s: %f (%f)" % (name, cv_results.
mean(), cv_results.std())
    print(msg)
```

Μετά την τυποποίηση δεδομένων, φαίνεται ότι ο αλγόριθμος SVM είχε την καλύτερη απόδοση (το αντίστοιχο αποτέλεσμα παρουσιάζεται και στο διάγραμμα).

Αφού προκύψει η λίστα αλγορίθμων μηχανικής μάθησης με καλό επίπεδο ακρίβειας, θα γίνει αξιοποίηση τρόπων βελτίωσης της ακρίβειας των μοντέλων. Συγκεκριμένα, θα γίνει αναζήτηση ενός συνδυασμού παραμέτρων για κάθε αλγόριθμο που αποδίδει τα καλύτερα αποτελέσματα και συνδυασμός της πρόβλεψης πολλαπλών μοντέλων σε μια πρόβλεψη συνόλου χρησιμοποιώντας αυτόνομους αλγόριθμους.

Παρακάτω ακολουθούν οι εντολές για την βελτίωση των αλγορίθμων KNN και SVM αντίστοιχα, καθώς και οι εντολές για την εκτύπωση των αποτελεσμάτων. Ο αλγόριθμος KNN φάνηκε να αποδίδει το χαμηλότερο σφάλμα όταν ο αριθμός των γειτόνων ισούται με 1. Ο αλγόριθμος SVM φάνηκε να αποδίδει το χαμηλότερο σφάλμα όταν η τιμή συντονισμού του C ισούται με 2.

```
scaler = StandardScaler().fit(X_train)
rescaledX = scaler.transform(X_train)
neighbors = [1,3,5,7,9,11,13,15,17,19,21]
param_grid = dict(n_neighbors=neighbors)
model = KNeighborsClassifier()
kfold = KFold(n_splits=num_folds, random_sta
te=seed)
grid = GridSearchCV(estimator=model, param_g
rid=param_grid, scoring=scoring, cv=kfold)
grid_result = grid.fit(rescaledX, y_train)
print("Best: %f using %s" % (grid_result.bes
t_score_, grid_result.best_params_))
means = grid_result.cv_results_['mean_test_s
core']
stds = grid_result.cv_results_['std_test_sco
re']
params = grid_result.cv_results_['params']
ranks = grid_result.cv_results_['rank_test_s
core']
for mean, stdev, param, rank in zip(means, s
tds, params, ranks):
    print("#%d %f (%f) with: %r" % (rank, me
an, stdev, param))
scaler = StandardScaler().fit(X_train)
rescaledX = scaler.transform(X_train)
c_values = [0.1, 0.3, 0.5, 0.7, 0.9, 1.0, 1.
3, 1.5, 1.7, 2.0]
kernel_values = ['linear', 'poly', 'rbf', 's
igmoid']
param_grid = dict(C=c_values, kernel=kernel_
values)
model = SVC()
kfold = KFold(n_splits=num_folds, random_sta
te=seed)
grid = GridSearchCV(estimator=model, param_g
rid=param_grid, scoring=scoring, cv=kfold)
grid_result = grid.fit(rescaledX, y_train)
print("Best: %f using %s" % (grid_result.bes
t_score_, grid_result.best_params_))
means = grid_result.cv_results_['mean_test_s
core']
```

```
stds = grid_result.cv_results_['std_test_score']
params = grid_result.cv_results_['params']
ranks = grid_result.cv_results_['rank_test_score']
for mean, stdev, param, rank in zip(means, stds, params, ranks):
    print("#%d %f (%f) with: %r" % (rank, mean, stdev, param))
```

Μια άλλη προσέγγιση βελτίωσης της απόδοσης είναι η αξιοποίηση των μοντέλων συνόλων. Για αυτό θα γίνει χρήση τεσσάρων αλγορίθμων μηχανικής μάθησης. Συγκεκριμένα, οι μέθοδοι ενίσχυσης: AdaBoost (AB) και Gradient Boosting (GBM) και οι μέθοδοι ενθυλάκωσης: Τυχαία δάση (RF) και επιπλέον δέντρα (ET). Παρακάτω φαίνονται οι αντίστοιχες εντολές και στη συνέχεια γίνεται η τελική σύγκριση των αλγορίθμων.

```
ensembles = []
ensembles.append(('AB', AdaBoostClassifier()))
ensembles.append(('GBM', GradientBoostingClassifier()))
ensembles.append(('RF', RandomForestClassifier()))
ensembles.append(('ET', ExtraTreesClassifier()))
results = []
names = []
for name, model in ensembles:
    kfold = KFold(n_splits=num_folds, random_state=seed)
    cv_results = cross_val_score(model, X_train, y_train, cv=kfold, scoring=scoring)
    results.append(cv_results)
    names.append(name)
    msg = "%s: %f (%f)" % (name, cv_results.mean(), cv_results.std())
    print(msg)
```

Εφόσον καταλήξουμε σε ένα μοντέλο που πιστεύουμε ότι μπορεί να κάνει ακριβείς προβλέψεις για τα δεδομένα, μπορούμε τελικά να ολοκληρώσουμε.

Για αυτό το σύνολο δεδομένων, ο αλγόριθμος SVM φαίνεται να αποδίδει συνολικά τα καλύτερα αποτελέσματα. Θα γίνει προετοιμασία του τελικού μοντέλου εκπαιδεύοντάς το με ολόκληρο το σύνολο δεδομένων εκπαίδευσης και θα εφαρμόσουμε τον ίδιο μετασχηματισμό δεδομένων στο σύνολο δεδομένων επικύρωσης.

Εντολές για προετοιμασία του μοντέλου

```
scaler = StandardScaler().fit(X_train)
rescaledX = scaler.transform(X_train)
model = SVC(C=1.5)
model.fit(rescaledX, y_train)
```

Εντολές για μεταμόρφωση του συνόλου δεδομένων επικύρωσης και για προβλέψεις

```
rescaledTestX = scaler.transform(X_test)
predictions = model.predict(rescaledTestX)
print(accuracy_score(y_test, predictions))
print(confusion_matrix(y_test, predictions))
print(classification_report(y_test, predictions))
print("1 refers to rocks, 0 refers to mines")
```

5 Συμπεράσματα

Η ακρίβεια στο σύνολο επικύρωσης παρατηρούμε ότι είναι περίπου 85.7%, δηλαδή είναι αρκετά κοντά στις αρχικές εκτιμήσεις που περιγράφονται στο αρχείο “sonar.names”. Συνεπώς, μπορούμε να καταλήξουμε στο συμπέρασμα ότι η ακρίβεια όσον αφορά την απόδοση των αλγορίθμων προκύπτει εξαιτίας της γενίκευσης των μοντέλων που χρησιμοποιήθηκαν σε σχέση με την αρχική δημοσίευση, καθώς έγινε χρήση ενός πολύ μεγαλύτερου σετ δοκιμών και ενός συνόλου απολύτως κατάλληλο για την εγκυρότητα των αποτελεσμάτων.

6 Πηγές/Αναφορές

[1] NOAA. What is sonar? National Ocean Service website, https://oceanservice.noaa.gov/facts/sonar.html, 25/06/18.

[2] Revisiting Machine Learning Datasets - Sonar, Mines vs. Rocks. 23/08/18. https://www.simonwenkel.com/2018/08/23/revisiting_ml_sonar_mines_vs_rocks.html

[3] Sonar Mines vs Rocks. https://www.kaggle.com/mattcarter865/sonar-mines-vs-rocks

[4] David Lowe. Simple Classification Model for the Sonar Dataset. 28/02/18. https://github.com/daines-analytics/tabular-data-projects/blob/master/py-classification-sonar-model/py-classification-sonar-model.ipynb

[5] David Lowe. Binary Classification Deep Learning Model for the Sonar Dataset Using Keras. 14/10/19. https://github.com/daines-analytics/deep-learning-projects/blob/master/py-keras-binary-classification-sonar-model/py-keras-binary-classification-sonar-model.ipynb

[6] Machine Learning Repository. Sonar, Mines vs. Rocks Dataset. https://archive.ics.uci.edu/ml/datasets/Connectionist+Bench+(Sonar,+Mines+vs.+Rocks)